Competitive Programming

Maximum Matching in Bipartite Graph

NCSU CS PhD student: Feifei Qiao

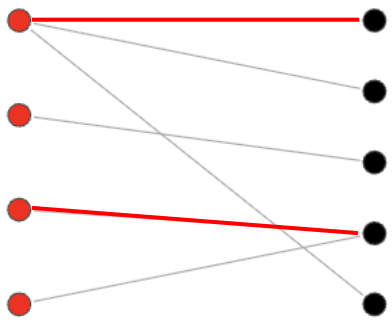
Bipartite Graph

1. A **bipartite graph** is a graph whose vertices can be divided into two disjoint sets:

$$V = L \cup R$$

such that:

Every edge connects a vertex in L to a vertex in R



2. Matching Problem:

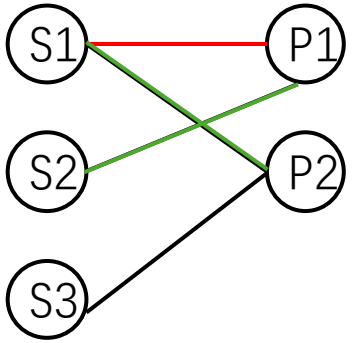
1) A **matching** is a set of edges such that: No two edges share a common vertex.

2) Each vertex appears in **at most one matched edge**.

3. Maximum Matching in Bipartite Graph:

A **maximum matching** is a matching with the **largest possible number of edges**.

Bipartite Graph



4. Augmenting Path

S2----P1 (unmatched)

P1----S1 (matched)

S1----P2 (unmatched)

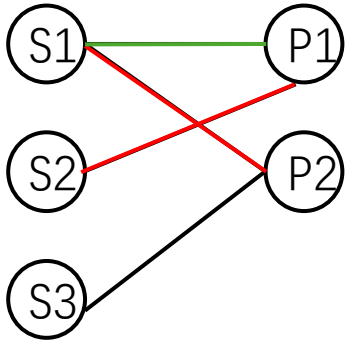
Augmenting Path: S2---P1---S1---P2

Reverse:

S2----P1 (matched)

P1----S1 (unmatched)

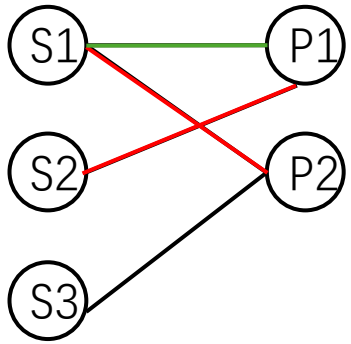
S1----P2 (matched)



The matching size increases by one

$Bipartite\ Graph$

5. Hungarian Algorithm



Step1: Initialize

- Let L and R be the two partitions.
- Initialize matching:

$$match[v] = -1\ \forall v \in R$$

- No vertices are matched initially.

Step 2: Iterate over Nodes in L

For each vertex $u \in L$:

- Try to find a matching for u
- Reset the visited array before each search:

$$visited[v] = false, \forall v \in R$$

Matching Array:

R	L
S1	P2
S2	P1
S3	-1

$Bipartite\ Graph$

Step 3 — Search for Augmenting Path (DFS)

For each neighbor v in R of u in L:

- If v has not been visited, mark v as visited
  - If v is unmatched (Augmenting path: u—(unmatched)-->v)
  - Or the node u' currently matched with v can be reassigned

(Augmenting path: u—(unmatched)-->v--(matched)-->u'--(unmatched)--> v'
reverse: u—(matched)-->v--(unmatched)-->u'--(matched)-> v')

Then:

$$match[v] = u$$

- Return success

Step 4 — Update Matching

- If DFS succeeds, increase matching size by 1
- Move to the next node in L

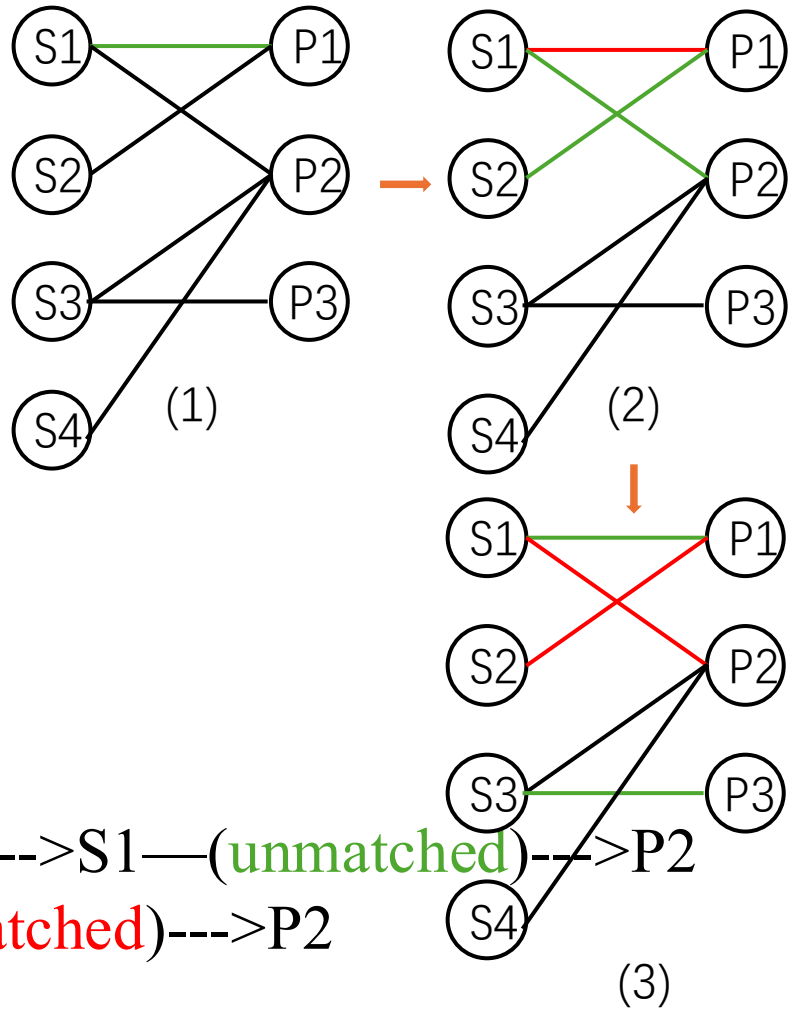
Step 5 — Repeat

Repeat the process for all nodes in L

Step 6 — Termination

When no more augmenting paths can be found: matching is maximum

Bipartite Graph



Step1: Initialize

match=[-1,-1,-1] for R

Step 2: Iterate over Left Nodes

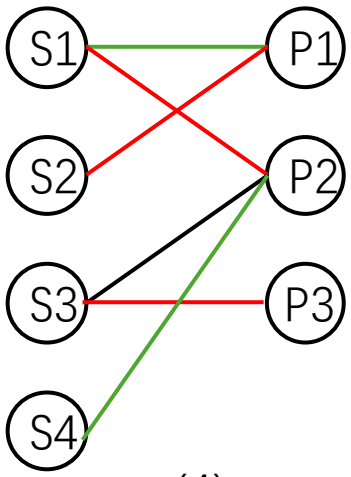
For each vertex s ∈ *L*:

- S1 (unmatching): S1->P1
  augmenting path: S1->P1
  update match=[S1,-1,-1]

- S2 (unmatching): S2->P1 (occupied by S1)
  augmenting path: S2—(unmatched)--->P1—(matched)--->S1—(unmatched)--->P2
  reverse: S2—(matched)--->P1—(unmatched)--->S1—(matched)--->P2
  update match = [S2, S1, -1]

- S3 (unmatching): S3->P2 (occupied by S1), can not reassign (because P1 occupied by S2), S3->P3 (free),
  augmenting path: S3->P3
  update match = [S2, S1, S3]

Bipartite Graph



(4)

- S4 (unmatching): S4->P2 (occupied by S1)

  attempt path:

S4-(unmatched)->P2-(matched)->S1-(unmatched)->P1-(matched)->S2
(failed because S2 does not have unmatched neighbor)

Step 3: No augmenting path found (**Termination**)

Maximum matching size: 3

Augmenting Paths Summary:

S1: S1 → P1
S2: S2 → P1 → S1 → P2
S3: S3 → P3
S4: no augmenting path